

TEMPO: Semantic-Action Decoupled RL Post-Training for Vision-Language-Action Models

Abstract—Vision-language-action (VLA) models are commonly adapted to downstream manipulation tasks via supervised fine-tuning (SFT) or online reinforcement learning (RL) post-training. SFT is prone to distribution mismatch, and existing RL approaches typically apply a single, uniform update strategy to all model components, ignoring their distinct functional roles. We propose TEMPO, a semantic-action decoupled, two-timescale RL post-training framework for VLA models. TEMPO freezes the pretrained vision-language backbone to preserve general semantic representations, and restricts adaptation to two components with dedicated RL optimization loops: the semantic projection layer and the low-level action expert. We update them at different rates—the semantic projection layer infrequently, to keep the latent action stable, and the action expert frequently, to rapidly incorporate control feedback from online interaction. This decoupling RL fine-tuning strategy prevents fast policy updates from destabilizing high-level semantic representations while still allowing the action expert to learn efficiently from online feedback. Experiments on the CALVIN benchmark and real-world manipulation tasks demonstrate that TEMPO consistently outperforms both pretrained state-of-the-art VLA models and the RL post-training baseline, while reaching and maintaining higher late-stage evaluation rewards on two real-world tasks.

I. INTRODUCTION

Vision-language-action (VLA) models extend multimodal foundation models beyond vision-language understanding to direct action generation, enabling robots to ground language instructions in visual observations and produce corresponding actions [1], [2], [3]. Large-scale robot datasets and open generalist policies further enable supervised pretraining across tasks, objects, scenes, and embodiments [4], [5], [6], allowing pretrained VLA models to serve as strong initial policies for instruction-conditioned manipulation.

Pretrained VLA models are commonly adapted to downstream robotic tasks through supervised fine-tuning (SFT) on demonstration trajectories, using either full-parameter fine-tuning or parameter-efficient methods such as LoRA [6], [7]. Although effective, SFT is inherently limited by offline data: high-quality robot trajectories are expensive to collect and cannot exhaustively cover the states encountered during deployment. More fundamentally, SFT optimizes action prediction under the demonstration distribution rather than closed-loop success under the policy-induced state distribution. Small execution errors can therefore compound over long-horizon tasks and eventually lead to failure [8]. These limitations motivate online post-training paradigms that enable policies to adapt through task-specific feedback acquired from their own interactions with the environment.

Online reinforcement learning (RL) [9], [10], [11] directly optimizes task rewards through environment interaction, en-

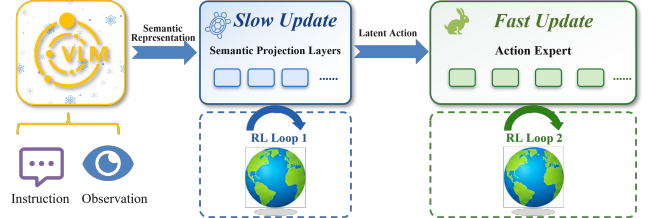


Fig. 1. **Method overview.** TEMPO keeps the VLM frozen and decouples the reinforcement learning update frequencies of the semantic projection layer and action expert, updating the former less frequently and the latter more frequently.

abling VLA policies to learn from the outcomes of their own executions. However, existing methods typically optimize the trainable components of a VLA policy under a unified RL update strategy, without considering the distinct functional roles and adaptation requirements of individual modules. In modular VLA architectures, the semantic representation layer and the action generation module serve different purposes and may require separate adaptation processes during online post-training. Therefore, effective RL post-training should decouple the optimization of these components and allow each module to adapt according to its own learning goals.

On the semantic side, directly updating a large vision-language backbone not only increases computational cost but may also disturb its pretrained representations [12], [13]. We therefore preserve the internet-scale semantic knowledge encoded by the pretrained vision-language backbone and fine-tune the downstream semantic projection layer in a dedicated RL loop, as it still retains task-relevant information and serves as the semantic-side optimization target. On the action side, the action expert directly maps latent actions to executable action chunks, and we adapt it in a separate RL loop to incorporate control- and execution-related feedback collected through online interaction.

After selecting the semantic projection layer and action expert as adaptation targets, a remaining question is how to coordinate their update frequencies. Frequent updates to the semantic projection layer may exacerbate drift in the latent action, whereas the action expert must incorporate interaction feedback sufficiently quickly to improve control. A uniform update schedule may therefore be unable to balance these distinct adaptation requirements.

We therefore propose TEMPO, a semantic-action decoupled reinforcement learning post-training framework, as illustrated in Fig. 1. TEMPO optimizes the semantic projection layer and the action expert through two separate module-level

TD3 [14] loops, while decoupling their update frequencies: the semantic projection layer is updated less frequently, whereas the action expert is updated more frequently. Our main contributions are summarized as follows:

- We propose TEMPO, a semantic–action decoupled RL post-training framework that optimizes different VLA components through dedicated RL loops. Specifically, TEMPO adapts the semantic projection layer while keeping the pretrained vision-language backbone frozen to preserve its large-scale semantic knowledge, and optimizes the action expert to learn control- and execution-related behaviors from online interaction feedback.
- We introduce a module-specific update-frequency strategy to coordinate the adaptation dynamics of semantic and action modules. TEMPO updates the semantic projection layer less frequently to mitigate latent-action drift, while updating the action expert more frequently to efficiently incorporate control feedback, achieving a balance between semantic stability and policy improvement.
- We evaluate TEMPO on CALVIN and two real-world manipulation tasks. The results show improved long-horizon performance over both state-of-the-art VLA models and the RL post-training baseline in simulation, as well as higher late-stage rewards in physical experiments.

II. RELATED WORK

A. Vision-Language-Action Models

VLA research has advanced primarily along two directions: scaling up robot datasets [15] and developing generalist policy architectures [16], [6]. Recent generalist models, such as π_0 [17], $\pi_{0.5}$ [18], and GR00T N1 [19] further extend VLA policies toward continuous action generation, open-world manipulation, and cross-embodiment control. These advances provide a stronger foundation for applying generalist VLA policies to specific robotic manipulation tasks. Building on these advances, supervised fine-tuning has become a common approach for adapting pretrained VLA policies to specific robots and tasks. Such methods typically update model parameters using task-specific demonstrations by either full-parameter fine-tuning [6] or parameter-efficient LoRA-based adaptation [7], while Octo [5] is designed for efficient adaptation to new sensory inputs and action spaces. These studies demonstrate that supervised fine-tuning can effectively improve VLA policy performance on specific tasks, but their optimization remains primarily driven by offline demonstration data.

Beyond general supervised fine-tuning strategies, recent studies have also begun to examine the interactions among functionally distinct VLA modules during adaptation. Knowledge insulation [12] investigates gradient interference between the vision-language backbone and the action expert, isolating action-side gradients to reduce their interference with pretrained vision-language knowledge. However, it mainly focuses on gradient isolation during supervised

training and does not consider separate RL optimization for different modules during online post-training or investigate the effects of relative update frequencies between semantic-side and action-side modules.

B. Reinforcement Learning for VLA Post-Training

To further improve pretrained VLA policies using on-line interaction feedback, existing RL post-training methods mainly focus on reward signals, interaction data, and optimization mechanisms. For reward signals, VLA-RL [9] uses a learned process reward, RIPT-VLA [20] adopts sparse binary success feedback, and VLA-RFT [21] employs simulator-verified rewards. For interaction data, ConRFT [22] uses real-robot intervention trajectories, whereas RECAP [23] combines cross-embodiment demonstrations, policy-generated experience, and human corrections. For optimization mechanisms, iRe-VLA [13] alternates between RL and supervised learning to alleviate training instability and computational overhead when updating large VLA models.

Beyond these designs, different methods expose distinct but predetermined parameter sets to RL optimization. VLA-RL [9] uses PPO to update the LoRA parameters of an autoregressive OpenVLA policy, whereas SimpleVLA-RL [10] applies GRPO to optimize all parameters of an action-token policy. iRe-VLA [13] keeps the vision-language backbone unchanged and updates only the action head during the RL stage, which is then alternated with supervised optimization over a broader set of model parameters. Overall, these methods typically fix the trainable modules before training or for each training stage. Their adaptation targets are selected mainly at the policy or stage level, with limited consideration of how functionally distinct modules may differ in their adaptation to interaction feedback.

C. Asynchronous VLA Execution and Optimization

Recent asynchronous VLA studies coordinate VLA components that operate at different frequencies across inference, modality refresh, and system execution. Acting While Understanding [24] refreshes reusable semantic conditions less frequently than action generation during inference, thereby reducing redundant semantic computation. DAM-VLA [25] maintains separate latent-representation buffers for different modalities. Each buffer is refreshed at its sensor’s native frequency and continuously read by the action head to support high-frequency control. RL-VLA³ [26] asynchronously coordinates environment interaction, model inference, and policy optimization to improve system throughput during RL post-training.

Overall, existing studies have explored VLA model design, RL post-training, and asynchronous inference and system optimization. However, the parameter-update frequencies of functionally distinct modules during RL post-training remain underexplored, particularly mechanisms that assign different update frequencies to semantic-side and action-side modules.

III. METHOD

A. Problem Formulation

We consider reinforcement learning post-training of a pre-trained VLA policy for robot manipulation. The interaction between the robot and the environment is modeled as a Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$, where $\mathcal{S}$ and $\mathcal{A}$ denote the state and action spaces, p denotes the environment transition dynamics, r is the task reward function, and γ is the discount factor. We formulate the interaction at the action-chunk level, and $\mathcal{A}$ therefore denotes the action-chunk space. At each step t , the robot receives visual observation o_t and language instruction l_t . Given the state $s_t = (o_t, l_t)$, the policy generates an action chunk $\mathbf{a}_t \in \mathcal{A}$, which is executed in the environment. The environment provides a sparse reward only upon episode termination, denoted by r_t . The RL post-training objective is

$$\max_{\omega} \mathbb{E}_{\tau \sim \pi_{\omega}} \left[\sum_{t=0}^{H-1} \gamma^t r_t \right], \quad (1)$$

where ω collectively denotes the trainable parameters of the VLA components selected for RL adaptation, τ denotes the closed-loop trajectory segment of the current active instruction, and H is the number of policy decision steps before that instruction terminates.

B. System Framework

We instantiate the pretrained VLA policy with FLOWER [27]. As illustrated in Fig. 2, the policy consists of a vision-language backbone VL, a semantic projection layer π_{θ}^s , and an action expert π_{ϕ}^a . Given the visual observation o_t and the language instruction l_t , the VLA policy is decomposed as

$$h_t = \text{VL}(o_t, l_t), \quad z_t = \pi_{\theta}^s(h_t), \quad \mathbf{a}_t = \pi_{\phi}^a(z_t), \quad (2)$$

where h_t denotes the multimodal semantic representation, z_t is the latent action provided to the action expert, and $\mathbf{a}_t$ is the robot action chunk.

We construct two separate module-level TD3 [14] learning loops around this architecture, each with its own replay buffer, twin critics, target networks, and optimizer. The framework decomposes RL post-training into two dedicated RL loops optimizing the semantic projection layer π_{θ}^s and the action expert π_{ϕ}^a , while the pretrained vision-language backbone remains frozen. θ and ϕ denote the trainable parameters of π_{θ}^s and π_{ϕ}^a , and are collectively represented as $\omega = (\theta, \phi)$.

For the *semantic-level* RL loop, the state space $\mathcal{H}$ consists of the multimodal semantic representations h_t , while the action space $\mathcal{Z}$ consists of the latent actions $z_t = \pi_{\theta}^s(h_t)$. Accordingly, the semantic-level state-action critic $Q^s(h_t, z_t)$ evaluates and updates π_{θ}^s . For the *action-level* RL loop, the state space is $\mathcal{Z}$, with z_t serving as the state, while the action space $\mathcal{A}$ consists of the action chunks $\mathbf{a}_t = \pi_{\phi}^a(z_t)$ executed in the environment. Similarly, the action-level state-action critic $Q^a(z_t, \mathbf{a}_t)$ evaluates and updates π_{ϕ}^a . Both loops share the same sparse reward $r_t = 1$ if the current task

is successfully completed after executing $\mathbf{a}_t$, and $r_t = 0$ otherwise.

The same interaction process generates the transition data required by both learning loops. Each interaction transition is used to construct a semantic-level transition for updating Q^s and π_{θ}^s , and an action-level transition for updating Q^a and π_{ϕ}^a . The latent action z_t produced by the semantic projection layer serves as the input to the action expert, while the two modules are updated through their respective critics and gradient pathways. This design allows the semantic projection layer and the action expert to be optimized in their corresponding learning spaces without direct gradient propagation between the two update pathways. The semantic projection layer should remain comparatively stable to limit drift^{what is limit drift?} in the latent action, whereas the action expert should adapt its control behavior in response to online interaction feedback. We therefore update the action expert at a higher frequency and the semantic projection layer at a lower frequency, as detailed in Section III-E.

C. Semantic-Level TD3

The semantic-level replay buffer stores

$$\mathcal{D}_S = \{(h_t, z_t, r_t, h_{t+1}, d_t)\}, \quad (3)$$

where d_t indicates whether the current active instruction terminates after executing $\mathbf{a}_t$. Bellman bootstrapping^{reference} is stopped at each subtask^{episode?} boundary, even when execution subsequently continues with the next instruction in the long-horizon chain.

Let the twin semantic-level critics be denoted by Q_1^s and Q_2^s , their target networks by $\bar{Q}_i^s$, and the target semantic projection layer by π_{θ}^s or $\bar{\pi}_{\theta}^s$. The target latent action is generated as

$$\tilde{z}_{t+1} = \pi_{\theta}^s(h_{t+1}). \quad (4)$$

Using the target latent action, the semantic-level Bellman target is defined as

$$y_t^S = r_t + \gamma(1 - d_t) \min_{i=1,2} \bar{Q}_i^s(h_{t+1}, \tilde{z}_{t+1}). \quad (5)$$

The twin semantic-level critics are optimized by minimizing the squared Temporal-Difference (TD) error:

$$\mathcal{L}_{Q^s} = \sum_{i=1}^2 \mathbb{E}_{\mathcal{D}_S} \left[(Q_i^s(h_t, z_t) - y_t^S)^2 \right]. \quad (6)$$

The semantic projection layer is optimized using the first semantic-level critic:

$$\mathcal{L}_s = -\mathbb{E}_{h_t \sim \mathcal{D}_S} [Q_1^s(h_t, \pi_{\theta}^s(h_t))]. \quad (7)$$

During this update, the replayed h_t is treated as a fixed input. Therefore, gradients are applied only to the semantic projection parameters θ and do not propagate into the frozen vision-language backbone or the action expert.

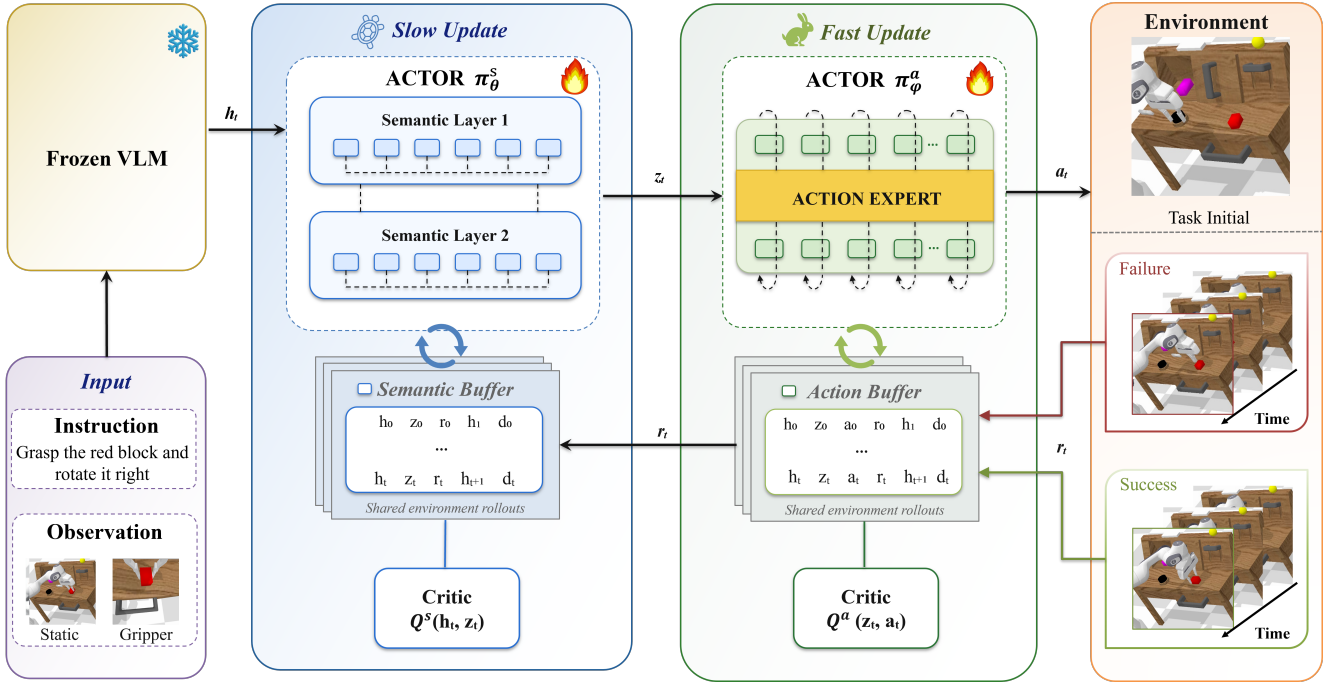


Fig. 2. **Framework of TEMPO.** The frozen VLM encodes the instruction and observation into h_t , which is transformed by the semantic projection layer into the latent action z_t and subsequently mapped by the action expert to the robot action chunk a_t . Two separate module-level TD3 loops, each equipped with a dedicated replay buffer and critic, optimize the two modules using shared environment rollouts, with a lower update frequency for the semantic projection layer and a higher one for the action expert.

D. Action-Level TD3

The action-level replay buffer stores

$$\mathcal{D}_A = \{(h_t, z_t, \mathbf{a}_t, r_t, h_{t+1}, d_t)\}, \quad (8)$$

where d_t follows the same active-instruction terminal definition as in the semantic-level replay buffer. The stored z_t is used for action-level critic updates, while the stored h_t allows the current latent action to be recomputed during action-expert updates.

Let the twin action-level critics be denoted by Q_1^a and Q_2^a , their target networks by $\bar{Q}_i^a$, the target semantic projection layer by π_θ^s , and the target action expert by π_ϕ^a . The target latent action and target action chunk are generated as

$$\tilde{z}_{t+1} = \pi_\theta^s(h_{t+1}), \quad \tilde{\mathbf{a}}_{t+1} = \pi_\phi^a(\tilde{z}_{t+1}). \quad (9)$$

Using the target latent action and action chunk, the action-level Bellman target is defined as

$$y_t^A = r_t + \gamma(1 - d_t) \min_{i=1,2} \bar{Q}_i^a(\tilde{z}_{t+1}, \tilde{\mathbf{a}}_{t+1}). \quad (10)$$

The twin action-level critics are optimized by minimizing the squared temporal-difference error:

$$\mathcal{L}_{Q^a} = \sum_{i=1}^2 \mathbb{E}_{\mathcal{D}_A} \left[(Q_i^a(z_t, \mathbf{a}_t) - y_t^A)^2 \right]. \quad (11)$$

For the action-expert update, the current latent action is recomputed from the replayed h_t as $z_t^{\text{sg}} = \text{sg}(\pi_\theta^s(h_t))$, where $\text{sg}(\cdot)$ denotes the stop-gradient operation. The action expert is then optimized using the first action-level critic:

$$\mathcal{L}_a = -\mathbb{E}_{h_t \sim \mathcal{D}_A} [Q_1^a(z_t^{\text{sg}}, \pi_\phi^a(z_t^{\text{sg}}))]. \quad (12)$$

During this update, z_t^{sg} is treated as a fixed input. Therefore, gradients are applied only to the action-expert parameters ϕ and do not propagate into the semantic projection layer or the frozen vision-language backbone.

E. Semantic-Action Frequency-Decoupled Optimization

We further decouple the update frequencies of the semantic projection layer and the action expert. Let N_a and N_s denote the numbers of action-level and semantic-level critic updates performed in each reinforcement learning post-training round, respectively. Both modules employ delayed actor updates. Specifically, the action expert is updated every d_a action-level critic updates, whereas the semantic projection layer is updated every d_s semantic-level critic updates. Their effective update frequencies per post-training round are therefore

$$f_a = \frac{N_a}{d_a}, \quad f_s = \frac{N_s}{d_s}, \quad (8)$$

where f_a and f_s denote the effective update frequencies of the action expert and the semantic projection layer, respectively. The action-to-semantic update-frequency ratio is then defined as

$$\rho = \frac{f_a}{f_s} = \frac{N_a d_s}{N_s d_a}. \quad (9)$$

When $\rho = 1$, the two modules are updated at the same frequency; when $\rho > 1$, the action expert is updated more frequently than the semantic projection layer. Adjusting ρ controls their relative update frequencies. We find that settings with $\rho > 1$ generally outperform $\rho = 1$, while different frequency ratios yield different levels of improvement.

IV. EXPERIMENTS

We conduct experiments to answer the following questions: (1) Does TEMPO improve long-horizon VLA performance over the pretrained state-of-the-art VLA policy and the RL post-training baseline? (2) Do the semantic projection layer and action expert provide complementary benefits during RL post-training? (3) How does the update-frequency ratio between the semantic project layer and the action expert affect performance? (4) How does the post-training task-set size affect performance?

A. Experimental Settings

Benchmark. We evaluate on the CALVIN ABC→D benchmark [28], a long-horizon language-conditioned manipulation setting in which policies are trained on environments A, B, and C and evaluated in the unseen environment D. Each rollout contains a chain of five natural-language instructions.

Metrics. Following the standard CALVIN protocol, we report the success rates of completing the first k tasks in a five-instruction chain, where $k = 1, \dots, 5$. In Table I, columns 1–5 correspond to these consecutive completion rates. Avg. Len. is the average number of consecutively completed tasks from the beginning of each chain.

Baselines. We compare our method with several representative language-conditioned multi-task policies on CALVIN. **RT-1** [2] is a scalable Transformer policy for end-to-end language-conditioned control. **GR-1** [29] leverages large-scale video generative pretraining for robotic manipulation. π_0 [17] generates continuous actions through flow matching, while $\pi_{0.5}$ [18] further incorporates heterogeneous co-training to improve open-world generalization. **UNIVLA** [30] integrates multimodal understanding and action generation within a unified generalist policy. **DeFI** [31] learns robot control by separating forward- and inverse-dynamics pretraining. **FLOWER** [27] is an efficient vision-language-flow model that generates continuous robot actions through conditional flow. ~~All reinforcement learning variants~~ **We only have one RL baseline.** ~~are initialized from the same pretrained FLOWER policy and subsequently undergo reinforcement learning post-training.~~ **FLOWER-RL** serves as our single-loop RL post-training baseline. It treats the semantic projection layer and the action expert as a composite actor and jointly updates them through a single TD3 loop. In contrast, **TEMPO** optimizes the two modules through two separate module-level TD3 loops and independently controls their update frequencies.

B. Simulation Results

Main Results. Our method achieves the best overall long-horizon performance under the CALVIN ABC→D setting. As shown in Table I, it ranks first across SR1–SR5 and Avg. Len., achieving an SR5 of 81.7% and an Avg. Len. of 4.59. Compared with DeFI, the strongest baseline in the table, our method improves SR5 and Avg. Len. by 0.5 percentage points and 0.08, respectively.

TABLE I: CALVIN ABC→D EXPERIMENTAL RESULTS.

Method	1	2	3	4	5	Avg. Len.
RT-1 [2]	53.3	22.2	9.4	3.8	1.3	0.90
RoboFlamingo [16]	82.4	61.9	46.6	33.1	23.5	2.47
SuSIE [32]	87.0	69.0	49.0	38.0	26.0	2.69
GR-1 [29]	85.4	71.2	59.6	49.7	40.1	3.06
3D Diffuser Actor [33]	92.2	78.7	63.9	51.2	41.2	3.27
OpenVLA [6]	91.3	77.8	62.0	52.1	43.5	3.27
CLOVER [34]	96.0	83.5	70.8	57.5	45.4	3.53
RoboDual [35]	94.4	82.7	72.1	62.4	54.4	3.66
π_0 [17]	93.8	85.0	76.7	68.1	59.9	3.84
$\pi_{0.5}$ [18]	92.7	84.6	77.3	69.3	61.2	3.85
UP-VLA [36]	92.8	86.5	81.5	76.9	69.9	4.08
RoboVLMs [37]	98.0	93.6	85.4	77.8	70.4	4.25
Seer-Large [38]	96.3	91.6	86.1	80.3	74.0	4.28
VPP [39]	96.5	90.9	86.6	82.0	76.9	4.33
UNIVLA [30]	98.9	94.8	89.0	82.8	75.1	4.41
DeFI [31]	97.9	94.2	90.7	87.0	81.2	4.51
FLOWER [27]	99.4	95.8	90.7	84.9	77.8	4.49
FLOWER-RL	99.4	96.7	91.3	85.5	78.4	4.51
TEMPO (Ours)	100.0	97.1	92.9	87.1	81.7	4.59

TABLE II: COMPONENT ABLATION.

Variant	1	2	3	4	5	Avg. Len.
Ours (full)	100.0	97.1	92.9	87.1	81.7	4.59
w/o projection update	99.9	96.8	91.6	86.1	79.8	4.54
w/o expert update	99.9	97.2	92.2	85.6	79.6	4.55

Because our method post-trains a FLOWER policy, we further compare it with the official FLOWER results reported. Our method raises SR5 from 77.8% to 81.7% and Avg. Len. from 4.49 to 4.59. The gains at SR1–SR5 are 0.6, 1.3, 2.2, 2.2, and 3.9 percentage points, respectively. Moreover, the performance drop from SR1 to SR5 decreases from 21.6 percentage points for FLOWER to 18.3 percentage points for our method, indicating that the improvement becomes more pronounced as the instruction chain grows longer.

FLOWER-RL jointly updates the semantic projection layer and the action expert through a single TD3 loop, simultaneously changing the latent action and the action mapping. This may induce drift in the latent action while the action expert is adapting. In contrast, TEMPO updates the semantic projection layer less frequently and the action expert more frequently, limiting latent-action drift while enabling the action expert to incorporate online interaction feedback. The widening performance advantage as the instruction chain grows longer is consistent with the intended effect of semantic-action frequency-decoupled adaptation.

Component Ablation. Table II analyzes the contributions of the action expert and the semantic projection layer. The w/o projection update variant keeps the semantic projection layer fixed and updates only the action expert, whereas the w/o expert update variant keeps the action expert fixed and updates only the semantic projection layer. Both variants improve over the FLOWER reference policy, demonstrating that adapting either the semantic projection layer or the action expert can provide additional benefits under the sparse instruction-completion reward. However, neither variant reaches the performance of TEMPO, which achieves the highest SR5 and Avg. Len. among all variants. These results indicate that the semantic projection layer and the action

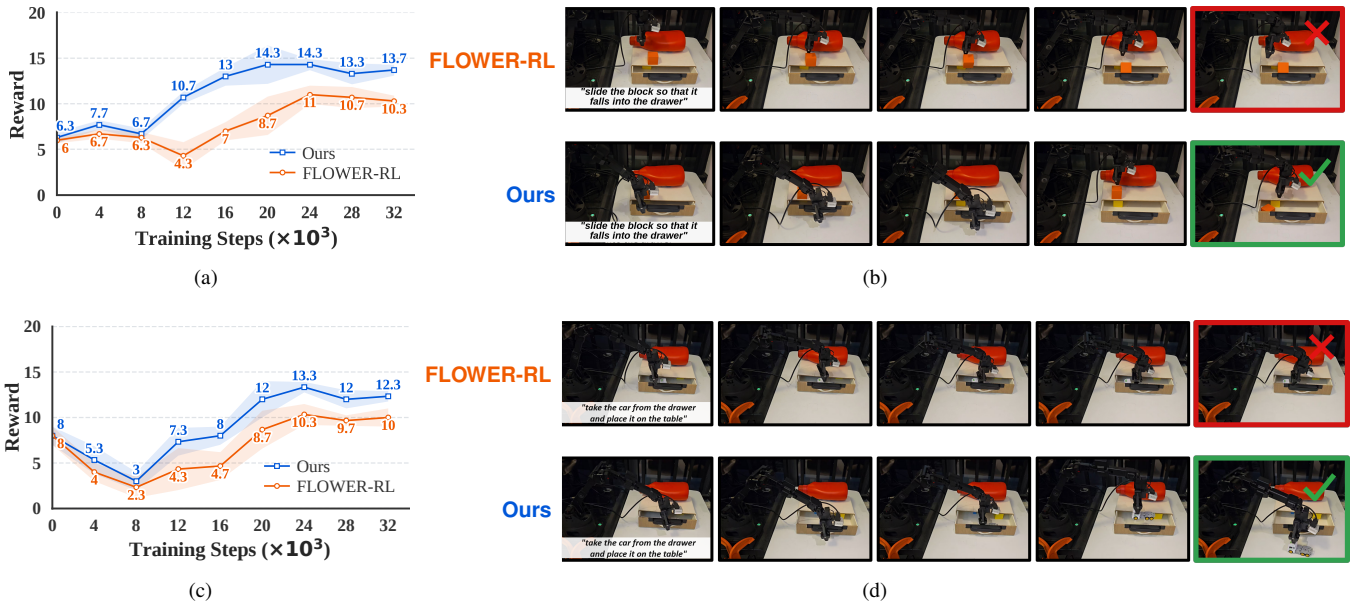


Fig. 3. **Real-world results on two manipulation tasks.** (a) and (b) show the block-sliding task, while (c) and (d) show the car-retrieval task. (a) and (c) report the evaluation reward during online RL post-training, with shaded regions denoting one standard deviation across three independent random seeds. (b) and (d) present representative execution sequences. For block sliding, FLOWER-RL directly manipulates the block without opening the drawer and fails, whereas TEMPO first opens the drawer and then slides the block inside. For car retrieval, FLOWER-RL fails to complete the retrieval and placement, whereas TEMPO sequentially opens the drawer, retrieves the car, and places it on the table.

TABLE III: ACTION-TO-SEMANTIC UPDATE-FREQUENCY RATIOS.

$f_a:f_s$	1	2	3	4	5	Avg. Len.
1:1	98.6	94.8	90.0	84.5	77.7	4.46
5:1	100.0	97.1	92.9	87.1	81.7	4.59
10:1	99.4	96.6	92.5	87.4	81.2	4.57

TABLE IV: RL POST-TRAINING TASK-SET SIZE COMPARISON.

RL task-set size	1	2	3	4	5	Avg. Len.
1	99.4	96.6	92.5	87.4	81.2	4.57
2	99.6	97.2	91.9	86.0	79.2	4.54
4	99.9	96.9	92.0	86.8	80.1	4.56
7	99.7	97.1	92.4	85.5	79.1	4.54
34	99.6	96.8	92.2	86.3	79.2	4.54

expert provide complementary adaptation capabilities. The similar performance of the two ablated variants suggests that neither module alone accounts for the full improvement.

Frequency-Decoupling Analysis. Table III evaluates different action-to-semantic update-frequency ratios $f_a : f_s$, where f_a and f_s denote the effective update frequencies of the action expert and the semantic projection layer, respectively. The 5:1 and 10:1 settings improve SR5 over FLOWER by 3.9 and 3.4 points, respectively, the equal-frequency 1:1 dual-loop setting is 0.1 point below FLOWER and reduces Avg. Len. from 4.49 to 4.46. These results indicate that the improvement does not arise from RL updates alone, but also depends on the relative update frequencies assigned to the two modules. A relatively higher update frequency for the action expert enables it to incorporate interaction feedback more frequently, while less frequent updates to the semantic projection layer keep the latent action comparatively stable. The synchronous 1:1 setting removes this frequency difference and substantially weakens the overall gain.

Post-Training Task-Set Size. Table IV compares different RL task-set sizes under the fixed 10:1 update-frequency ratio. Each rollout contains five subtasks, but only transitions associated with the selected task set are used for training. Thus,

the task-set size denotes the number of selected subtasks among CALVIN’s 34 subtasks, rather than the instruction-chain length.

All evaluated settings outperform the FLOWER reference policy, with the single-task setting achieving the highest SR5 and Avg. Len. However, performance does not improve monotonically as the task set expands: the full 34-task setting remains better than FLOWER but performs worse than the single-task setting. These results suggest that simply including more post-training tasks does not necessarily improve performance, and that the task composition and allocation of interaction data may require careful balancing.

C. Real-World Experiments

Experimental Setup. We conduct multi-stage language-conditioned manipulation experiments on a physical platform comprising a 6-DoF Synria Alicia-D robot arm and a single-DoF gripper. Visual observations are provided by two Intel RealSense cameras: a D435i captures a third-person RGB view of the workspace, while a wrist-mounted D405 records close-range observations around the gripper. We evaluate two multi-stage manipulation tasks, both of which require the robot to satisfy a prerequisite before interacting with the target object. For each task, we collect 60 human demonstration

episodes, resulting in 120 episodes in total. Each episode contains synchronized images, robot states, executed action commands, and language instructions. These demonstrations provide task-specific supervised data before online RL post-training.

Online RL Post-Training. Fig. 3(a) and (c) report the evaluation reward during online RL post-training. For each random seed, we evaluate 20 episodes at every checkpoint and sum the binary task rewards, assigning 1 to success and 0 to failure. Across both tasks, the evaluation rewards fluctuate during the early stage and then gradually improve and stabilize as post-training proceeds. Compared with FLOWER-RL, TEMPO reaches and maintains higher reward levels during the later stages of post-training.

Qualitative Analysis. Fig. 3(b) and (d) compare representative executions when the drawer is not sufficiently open. When the drawer is already fully open, FLOWER-RL and TEMPO achieve comparable performance. Their performance gap mainly emerges when the limited drawer opening leaves insufficient space for direct target-object manipulation. Through online RL post-training, TEMPO learns the implicit task semantics that the drawer should first be opened further and then completes the subsequent stages in the correct order. In contrast, FLOWER-RL directly attempts to manipulate the target object and fails. Its joint optimization of the semantic projection layer and action expert may induce drift in the latent action, making such implicit semantic relations more difficult to learn and preserve.

V. CONCLUSION

This paper presented TEMPO, a semantic-action frequency-decoupled RL post-training framework that uses two coupled module-level TD3 loops to adapt the semantic projection layer and the action expert while keeping the pretrained vision-language backbone frozen. On CALVIN, TEMPO outperforms FLOWER and FLOWER-RL in long-horizon metrics, while component and frequency ablations support the complementary value of adapting both modules and the importance of their relative update frequencies. On two real-world manipulation tasks, TEMPO reaches and maintains higher late-stage evaluation rewards than FLOWER-RL, while representative executions suggest more reliable prerequisite-first action ordering.

Future work will explore learning a language-conditioned dense reward function to provide richer intermediate feedback than the sparse instruction-completion reward. Such progress-aware signals may improve credit assignment and sample efficiency during RL post-training, especially for long-horizon tasks.

REFERENCES

- [1] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “PaLM-E: An embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 8469–8488. [Online]. Available: <https://proceedings.mlr.press/v202/driess23a.html>
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. S. Rycot, G. Salazar, P. R. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. H. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “RT-1: Robotics transformer for real-world control at scale,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. [Online]. Available: <https://roboticsproceedings.org/rss19/p025.html>
- [3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soric, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Rycot, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 06–09 Nov 2023, pp. 2165–2183. [Online]. Available: <https://proceedings.mlr.press/v229/zitkovich23a.html>
- [4] Open X-Embodiment Collaboration, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6892–6903. [Online]. Available: <https://ieeexplore.ieee.org/document/10611477>
- [5] D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, Q. Vuong, T. Xiao, P. R. Sanketi, D. Sadigh, C. Finn, and S. Levine, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. [Online]. Available: <https://www.roboticsproceedings.org/rss20/p090.html>
- [6] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2679–2713. [Online]. Available: <https://proceedings.mlr.press/v270/kim25c.html>
- [7] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. [Online]. Available: <https://www.roboticsproceedings.org/rss21/p017.html>
- [8] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 15. PMLR, 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>
- [9] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, “VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.18719>
- [10] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, D. Wang, D. Luo, Y. Fan, Y. Sun, J. Zeng, J. Pang, S. Zhang, Y. Wang, Y. Mu, B. Zhou, and N. Ding, “SimpleVLA-RL: Scaling VLA training via reinforcement learning,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=TQhSodCM4r>
- [11] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke, “RL token: Bootstrapping online rl with vision-language-action models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [12] D. Driess, J. T. Springenberg, B. Ichter, L. Yu, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi, and S. Levine, “Knowledge insulating vision-language-action models: Train fast, run fast, generalize better,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

- [Online]. Available: https://papers.nips.cc/paper_files/paper/2025/hash/94e936034d12bcd04834ec2773f02aff-Abstract-Conference.html
- [13] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen, "Improving vision-language-action model with online reinforcement learning," 2025, accepted to ICRA 2025. [Online]. Available: <https://arxiv.org/abs/2501.16664>
- [14] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 1587–1596. [Online]. Available: <https://proceedings.mlr.press/v80/fujimoto18a.html>
- [15] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. W. He, V. Myers, M. J. Kim, M. Du, A. Lee, K. Fang, C. Finn, and S. Levine, "BridgeData V2: A dataset for robot learning at scale," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 1723–1736. [Online]. Available: <https://proceedings.mlr.press/v229/walke23a.html>
- [16] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu, C. Cheang, Y. Jing, W. Zhang, H. Liu, H. Li, and T. Kong, "Vision-language foundation models as effective robot imitators," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=IFYj0oibGR>
- [17] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, L. Smith, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, " π_0 : A vision-language-action flow model for general robot control," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. [Online]. Available: <https://www.roboticsproceedings.org/rss21/p010.html>
- [18] K. Black, N. Brown, J. Darphinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, " $\pi_{0.5}$: A vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 27–30 Sep 2025, pp. 17–40. [Online]. Available: <https://proceedings.mlr.press/v305/black25a.html>
- [19] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Ljontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu, "GR00T N1: An open foundation model for generalist humanoid robots," 2025. [Online]. Available: <https://arxiv.org/abs/2503.14734>
- [20] S. Tan, K. Dou, Y. Zhao, and P. Krähenbühl, "Interactive post-training for vision-language-action models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.17016>
- [21] H. Li, P. Ding, R. Suo, Y. Wang, Z. Ge, D. Zang, K. Yu, M. Sun, H. Zhang, D. Wang, and W. Su, "VLA-RFT: Vision-language-action reinforcement fine-tuning with verified rewards in world simulators," 2025. [Online]. Available: <https://arxiv.org/abs/2510.00406>
- [22] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, "ConRFT: A reinforced fine-tuning method for VLA models via consistency policy," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. [Online]. Available: <https://www.roboticsproceedings.org/rss21/p019.html>
- [23] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darphinian, K. Dhabalia, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchii, M. Lamb, D. LeBlanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tanner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou, " $\pi_{0.6}^*$: a VLA that learns from experience," 2025. [Online]. Available: <https://arxiv.org/abs/2511.14759>
- [24] S. Yan, G. Wang, Q. Liu, W. Meng, J. Yang, C. Yao, F. Feng, X. Ma, Y. Zhao, and Y. Han, "Acting while understanding: Asynchronous semantic-action decoupling for real-time vision-language-action models," 2026. [Online]. Available: <https://arxiv.org/abs/2606.15285>
- [25] P. Vanjani, Z. Li, J. Suliga, M. Reuss, G. Geraci, X. Jiang, and R. Lioutikov, "DAM-VLA: Decoupled asynchronous multimodal vision language action model," 2026. [Online]. Available: <https://arxiv.org/abs/2606.12105>
- [26] H. Sun, Y. Guo, Z. Guan, S. Di, X. Bai, J. Long, T. Zhao, M. Luo, H. Zhao, L. Wu, X. Deng, X. Chu, X. Xiao, S. Wen, Y. Gong, and J. Xiong, "RL-VLA³: A flexible and asynchronous reinforcement learning framework for VLA training," 2026. [Online]. Available: <https://arxiv.org/abs/2602.05765>
- [27] M. Reuss, H. Zhou, M. Rühle, Ö. E. Yağmurlu, F. Otto, and R. Lioutikov, "FLOWER: Democratizing generalist robot policies with efficient vision-language-flow models," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 27–30 Sep 2025, pp. 3736–3761. [Online]. Available: <https://proceedings.mlr.press/v305/reuss25a.html>
- [28] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, "CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [29] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong, "Unleashing large-scale video generative pre-training for visual robot manipulation," 2023. [Online]. Available: <https://arxiv.org/abs/2312.13139>
- [30] Y. Wang, X. Li, W. Wang, J. Zhang, Y. Li, Y. Chen, X. Wang, and Z. Zhang, "Unified vision-language-action model," 2025. [Online]. Available: <https://arxiv.org/abs/2506.19850>
- [31] W. Zhang, B. Zhang, Z. Qi, W. Zeng, X. Jin, and L. Zhang, "Disentangled robot learning via separate forward and inverse dynamics pretraining," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=DdrsHWobR1>
- [32] K. Black, M. Nakamoto, P. Atreya, H. Walke, C. Finn, A. Kumar, and S. Levine, "Zero-shot robotic manipulation with pretrained image-editing diffusion models," 2023. [Online]. Available: <https://arxiv.org/abs/2310.10639>
- [33] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki, "3D diffuser actor: Policy diffusion with 3D scene representations," 2024. [Online]. Available: <https://arxiv.org/abs/2402.10885>
- [34] Q. Bu, J. Zeng, L. Chen, Y. Yang, G. Zhou, J. Yan, P. Luo, H. Cui, Y. Ma, and H. Li, "Closed-loop visuomotor control with generative expectation for robotic manipulation," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 139 002–139 029.
- [35] Q. Bu, H. Li, L. Chen, J. Cai, J. Zeng, H. Cui, M. Yao, and Y. Qiao, "Towards synergistic, generalized, and efficient dual-system for robotic manipulation," 2024. [Online]. Available: <https://arxiv.org/abs/2410.08001>
- [36] J. Zhang, Y. Guo, Y. Hu, X. Chen, X. Zhu, and J. Chen, "UP-VLA: A unified understanding and prediction model for embodied agent," 2025. [Online]. Available: <https://arxiv.org/abs/2501.18867>
- [37] X. Li, P. Li, L. Qian, M. Liu, D. Wang, J. Liu, B. Kang, X. Ma, X. Wang, D. Guo, *et al.*, "What matters in building vision-language-action models for generalist robots," *Nature Machine Intelligence*, pp. 1–15, 2026.
- [38] Y. Tian, S. Yang, J. Zeng, P. Wang, D. Lin, H. Dong, and J. Pang, "Predictive inverse dynamics models are scalable learners for robotic manipulation," 2024. [Online]. Available: <https://arxiv.org/abs/2412.15109>
- [39] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen, "Video prediction policy: A generalist robot policy with predictive visual representations," in *Proceedings of the 42nd International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=c0dhw1du33>